Two Phase Locking (2PL)

Two-phase locking (2PL) is a concurrency-control protocol that guarantees **conflict serializability**. Transactions can interleave, but their conflicting operations must be consistent with some serial order.

Two-phase locking is a **pessimistic** approach to concurrency control. It assumes that conflicts may occur, so transactions acquire locks before accessing data. These locks prevent conflicting operations from proceeding at the same time. The trade-off is that acquiring and releasing locks adds overhead, even when no conflict would have occurred.

In the basic model, a transaction adheres to the following:

1. A transaction takes a **shared lock** to read a data item (e.g. SELECT).
2. A transaction takes an **exclusive lock** to write a data item (e.g. INSERT, UPDATE, DELETE).
3. Multiple transactions can hold shared locks on the same item.
4. An exclusive lock prevents other transactions from acquiring either kind of lock on that item until it is released.

Lock Compatibility Matrix

	S	X
S	√	–
X	–	–

Rules of two-phase locking (2PL)

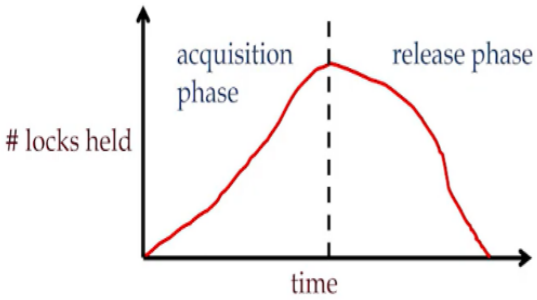
1. **Lock before accessing data.** A transaction needs a **shared (S)** to read an item and an **exclusive (X) lock** to write it (also can use it to read).
   - Multiple transactions can hold shared locks on the same item.
   - An exclusive lock blocks other transactions from acquiring either type of lock on that item.
2. **Growing (or Acquisition) phase.** The transaction can acquire new locks and upgrade shared locks to exclusive locks.
3. **Shrinking (or Release) phase.** The first lock release starts this phase. The transaction can use and release locks it still holds. It cannot acquire new locks or upgrade existing ones.

For example, `lock A → lock B → unlock A → unlock B` follows 2PL. The transaction acquires both locks before releasing either one. **Acquiring B** is the *lock point*, and **releasing A** starts the shrinking phase.

Basic 2PL guarantees conflict serializability.

The two phases:

Phase	What the transaction can do
Acquisition phase	Acquire locks. It cannot release any lock.
Release phase	Release locks. It cannot acquire any new lock.



The first lock release is the turning point. A transaction can acquire locks as it needs them, but once it releases its first lock, it must finish using the locks it already holds.

Why 2PL Guarantees Conflict Serializability

Two-phase locking (2PL) guarantees conflict serializability because **every edge in the dependency graph points from a transaction with an earlier lock point to one with a later lock point**. A transaction's **lock point** is the moment it acquires its last lock.

Suppose T1 writes A before T2 reads A. This conflict gives the edge **T1 → T2**. T1 holds an exclusive lock on A, so T2 must wait for T1 to release it before acquiring a shared lock on A.

```
T1 reaches its lock point
    → T1 releases its lock on A
    → T2 acquires its lock on A
    → T2 reaches its lock point
```

Under 2PL, T1 must acquire all its locks before releasing any. That puts T1's lock point before T2's. The same reasoning applies to every conflicting pair.

A cycle would require a transaction's lock point to come **before itself**, which is impossible. So the graph is acyclic, and ordering transactions by their lock points gives a **conflict-equivalent serial schedule**.

Cascading Aborts

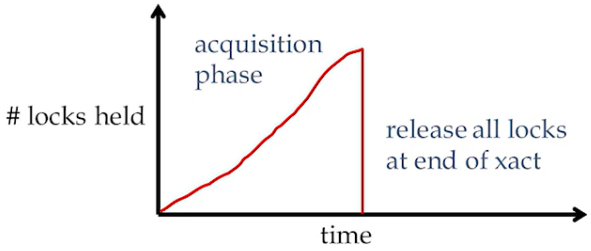
A **cascading abort** occurs when one transaction rolls back and other transactions must roll back because they read its uncommitted changes.

For example, if T2 reads a value written by T1 and T1 aborts, T2 must also abort because it used a value that was undone.

T1:	**R(A), W(A)**		**Abort**
T2:		**R(A), W(A)**	

Strict Two Phase Locking

Basic two-phase locking (2PL) allows a transaction to release locks before it commits or rolls back. Once it releases its first lock, it can't acquire any new locks or upgrade existing ones.



Strict two-phase locking (strict 2PL) follows the same rule and also holds every **exclusive (write) lock** until commit or rollback. Shared locks can be released earlier, subject to the basic 2PL rule.

For example, after T_1 writes A, basic 2PL may let it release the exclusive lock on A before committing. T_2 could then read that uncommitted value. Under strict 2PL, T_2's request for a shared lock on A waits until T_1 commits or rolls back, preventing a **dirty read**.

Non-2PL Example:

Non-2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A): (A=1000)	
	Lock_S(A)
A: = A-50 (A=950)	
Write(A) A=950	
Unlock(A)	
	Read(A) (A = 950)
	Unlock(A)
	Lock_S(B)
Lock_X(B)	
	Read(B) (B=2000)
	Unlock(B)
	PRINT(A), PRINT(B), PRINT(A+B)
Read(B) (B=2000)	
B := B +50 (B=2050)	
Write(B) B=2050	
Unlock(B)	

Output: 950, 2000, 2950

Output: 950, 2050, 3000

2PL Example:

2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A): (A=1000)	
A: = A-50 (A = 950)	
Write(A) A = 950	
Lock_X(B)	
Unlock(A)	
	Lock_S(A)
	Read(A): (A = 950)
Read(B) (B = 2000)	
B := B +50 (B = 2050)	
Write(B) B = 2050	
Unlock(B)	
	Lock_S(B)
	Unlock(A)
	Read(B) (B = 2050)
Output: 950, 2050, 3000	Unlock(B)
	PRINT(A), PRINT(B), PRINT(A+B)

Strict 2PL Example:

Strict 2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A): (A = 1000)	
	Lock_S(A)
A: = A-50 (A = 950)	
Write(A) A = 950	
Lock_X(B)	
Read(B) (B = 2000)	
B := B +50 (B = 2050)	
Write(B) B = 2050	
Unlock(A)	
Unlock(B)	
	Read(A) (A = 950)
	Lock_S(B)
	Read(B) (B = 2050)
	PRINT(A), PRINT(B), PRINT(A+B)
Output: 950, 2050, 3000	Unlock(A)
	Unlock(B)

Which schedules allow Strict 2PL

